

AI-Enhanced Matchmaking App for Language Exchange

DCC-6118

Tejas Khare, Maksym Kushniryk, Noah Martin, Bryant Morales-Chora, Patrisiya Rumyantseva

Client: Dr. Yongtaek Kim

Repository: <https://github.com/patrisiyarum/KoreanMatchmakingApp-DCC-6118/tree/main>

Table of Contents

Terminology	4
Introduction.....	5
System Architecture	6
Introduction	6
Rationale.....	6
Static System Architecture	7
Data Design	9
Introduction.....	9
Database Use	10
File Use.....	11
Data Exchange	12
Format and Protocols	12
Security Considerations	13

List of Figures

Figure 1: AI-Enhanced Matchmaking App Architecture	7
Figure 2: Entity Relation Diagram	10

Terminology

API (Application Programming Interface): A set of rules that allow different software systems to communicate and exchange data in a structured manner

CORS (Cross-Origin Resource Sharing): A security mechanism for controlling which origins can access the backend API.

HTTPS (HyperText Transfer Protocol Secure): A secure version of HTTP that encrypts data transmitted between a user's browser and a web server using SSL/TLS protocols to protect privacy.

MCP (Model Context Protocol): A protocol that enables an AI model to use tool-calls in an efficient and standardized way.

MySQL: An open-source relational database management system used to store, organize, and manage structured data through tables & SQL queries.

ORM (Object-Relational Mapping): A technique for querying and manipulating a database in an object-oriented way (JavaScript with Sequelize here).

REST (Representational State Transfer): A standard for designing APIs using HTTP methods for simple use.

CSS (Cascading Style Sheet): A stylesheet language used to control the presentation and layout of web pages, including colors, fonts, and spacing.

React: A JavaScript library for building user interfaces using reusable components and a virtual DOM for efficient rendering.

Introduction

Background

This application is an existing web-based language exchange platform created to connect university students learning Korean and English by facilitating partner matchmaking and AI-supported language practice. Users can create profiles, search for compatible partners, schedule sessions, and engage in live conversations. Our stack includes React.js (frontend), Node.js (backend), MySQL (database), and Gemini API (AI). The present iteration focuses on integrating new tools, including improved matchmaking flow, gamification elements, and a more intuitive UI design. These features aim to supplement the low-stakes conversational environment while maintaining opportunities for authentic peer interaction.

Document Summary

The System Architecture section describes the structural components (static) of the application and how they interact during runtime (dynamic). It explains how user actions, partner interactions, and AI features flow through the platform.

The Data Storage Design section defines how data is formatted, stored, and managed within the application. It details the organization of user profiles, conversations, feedback, and other central platform data to ensure scalability and reliability.

System Architecture

Introduction

The architecture of the AI-Enhanced Matchmaking App is designed to support the various features of this app to the best extent possible. Most fundamentally, it aims to support intelligent partner matching, which is the foundational purpose of the app. In addition, the system design must support other fundamental features such as real-time communication through both text-based chat (via Socket.io) and video conferencing (via Agora RTC SDK) for live language practice sessions. Supplemental features like an AI Language Assistant and session scheduling must also be supported, all while maintaining a privacy-conscious design that gives users control over what AI can and can't access.

Rationale

The language making app is designed with a **layered architecture** design pattern, where the system is organized into multiple different layers. Each layer takes input from the layer below it and provides output to the layer above it, which allows us to effectively tackle the challenges of supporting the wide array of functionality our app requires.

This design pattern was chosen for multiple reasons, including:

1. **Separation:** The layered architecture cleanly separates the different responsibilities that our app must support (storing data, handling business logic, handling HTTP routing, front-end presentation). This makes the system easier to develop and maintain.
2. **Parallelized Development:** Since each layer communicates with only adjacent layers through clearly defined communication patterns, the architecture enables the developer team to work simultaneously on multiple different layers at once. This greatly speeds up development as members can work in parallel without affecting one another's work.
3. **Flexibility:** Since each individual layer uses its own technology stack that does not affect the rest of the system, this architecture enables us to swap our technologies in individual layers cleanly without affecting the rest of the system. This aids us as we aim to find the best suited technologies for solving our problem.
4. **Testability:** A layered design allows the developers to test each layer in a clean, simple way, without worrying about the correctness of other layers. By testing all layers, we can prove the correctness of the entire app.
5. **Security:** The Service Layer hashes all passwords before they reach the Data Access Layer, allowing us to securely handle passwords by preventing the backend from ever storing or comparing them in plain text. No route can bypass this.

Static System Architecture

A UML Component Diagram was used for our static system architecture. It is the best UML diagram for this purpose because it allows us to see the major components of the system (each layer, along with the subcomponents of each layer), alongside how these components interact and depend on each other. This is beneficial compared to a class diagram, which would be too granular for our goal of demonstrating the functionality of the entire system. The component diagram depicts how our system architecture is logically organized, and reflects the intent of the layered architecture well.

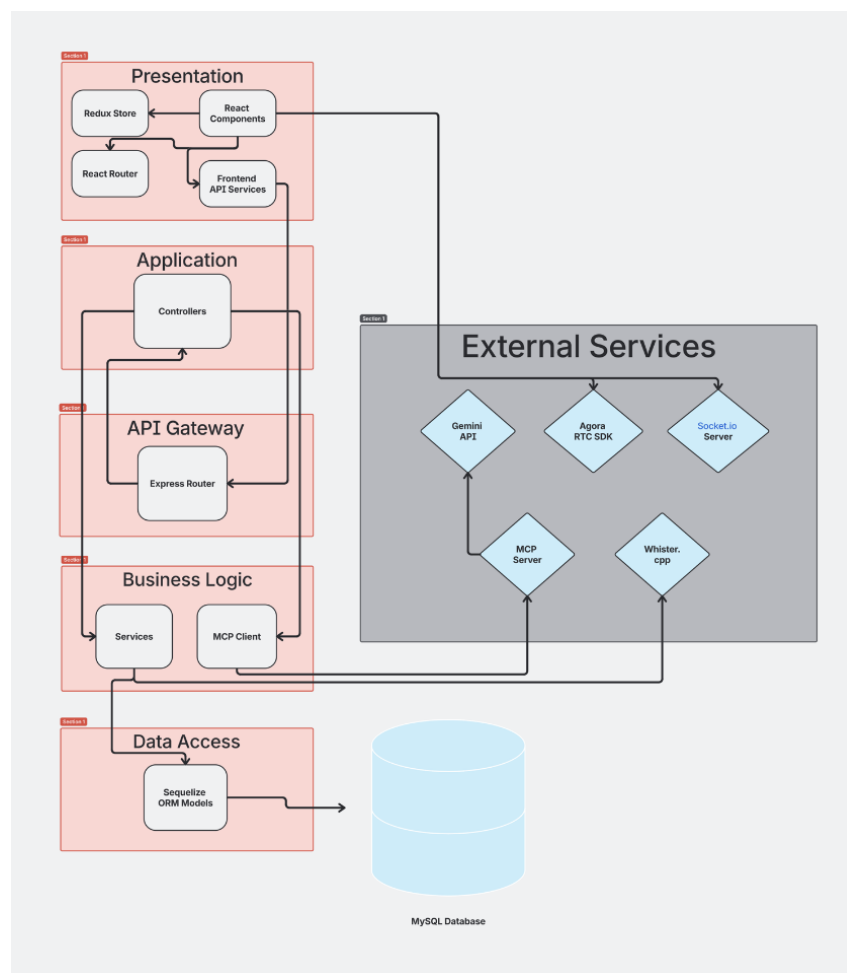


Figure 1: AI-Enhanced Matchmaking App Architecture

The AI-Enhanced Matchmaking App for Language Exchange has a six-layer static architecture. The React frontend with Redux state management, the React Router for navigation, and Axios-based service modules that interface with the backend are all part of the Presentation Layer. Additionally, this layer is directly dependent on the Socket.io Server for real-time chat and the

Agora RTC SDK for video conferencing. All HTTP requests are received by the API Gateway Layer (Express Router), which then forwards them to the Application Layer (Controllers), which arranges calls to the Business Logic Layer. Twelve service modules that manage core domain logic, such as user registration using bcrypt hashing and privacy enforcement, are part of the Business Logic Layer. Additionally, the MCP Client interacts with the MCP Server to activate AI-powered features (pronunciation assessment, partner matching, meeting scheduling, and session summarization) that use the Google Gemini API.

Sequelize ORM is used by the Data Access Layer, which sits beneath the Business Logic Layer, to define sixteen models that map to MySQL tables. Versioned migrations are used to abstract all database operations and schema evolution. All persistent application data, such as user accounts, profiles, friendships, chats, meetings, transcripts, and AI conversations, are stored in a MySQL database at the Database Layer's base.

Data Design

Introduction

The key aspect of data design is constructing an ER diagram, as it is what we will reference when creating our database. Figure 2 is the entity-relationship (ER) diagram for the Language Exchange Matchmaker website. This diagram visualizes how the website will store, organize, and exchange important information tied to the frontend, backend, and database systems. It will be used as an outline for creating our database, which will be implemented using MySQL. We are using MySQL as it will allow us to use queries to quickly retrieve and update data contained within the database. The entities in the diagram each contribute to a certain feature(s) of the website, which includes user accounts/profiles, voice calls, text messaging, scheduler, AI chat, and gamification.

Along with the ER diagram, this section looks at our system's file usage and data exchange. File usage is the different types of files we will need to use for backend, frontend, and database systems. There may also be some file types that are needed for other specific purposes, such as image files and config files. With data exchange, we will look at what data protocols we will use for data transfer between devices, as well as how we plan to keep user data secure.

Database Use

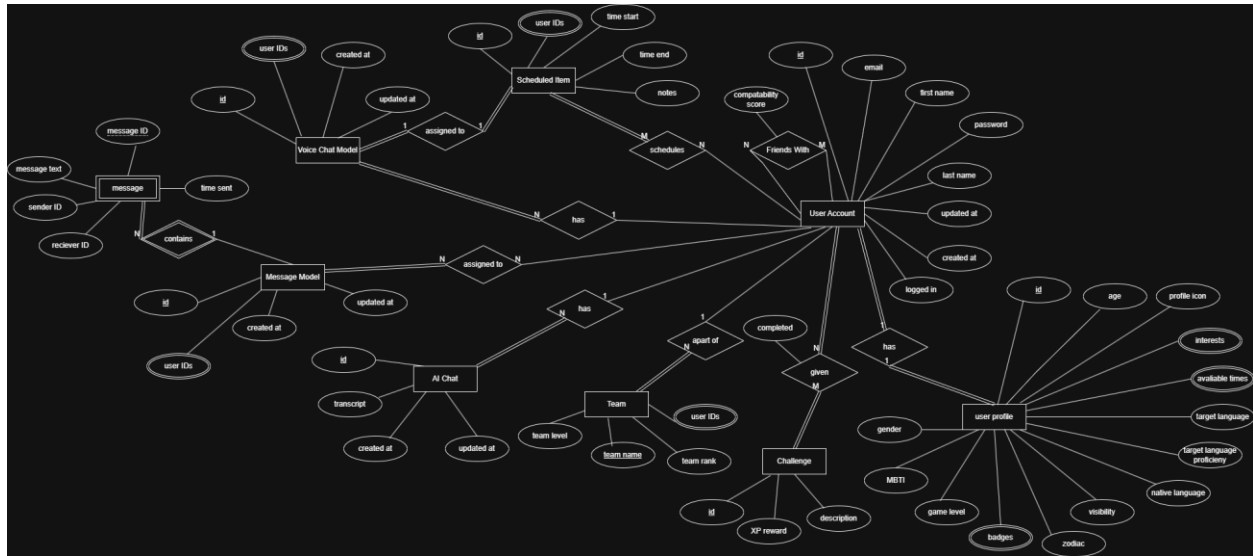


Figure 2: Entity Relation Diagram

Entity List:

User Account: Stores login information like email address, password, first name, and last name. Tracks when the account was created, last updated, and the last time they logged in. Also connect to other entities like profiles, chats, and friendships.

User Profile: Contains personal information of a user, such as age, gender, MBTI, zodiac sign, interests, game level, badges, and more. This information is needed for matchmaking and gamification purposes.

Voice Chat Model: Represents voice chat sessions between users. It contains information relating to the users' IDs, as well as when the session was created and last updated.

Scheduled Item: Represents when a call is scheduled to be for a pair of users. Contains data on the users' IDs, the time start/end of the scheduled call, and any notes about the scheduled item.

Message Model: Represents text message chat sessions between users. It contains information relating to the users' IDs, as well as when the session was created and last updated.

Message: Weak entity, meaning it cannot exist without being assigned to a message model. Holds information related to each individual message, which includes the sender/receiver IDs, the actual text within the message, and the time the message was sent.

AI Chat: Represents the interactions between a user and the AI system. Stores conversations and timestamps for when the AI chat was created and last updated.

Team: Represents a team, which essentially is a collective of users. Along with the team members' IDs, this entity stores data about the team level and team rank.

Challenge: Challenges get assigned to users to complete, giving them bonus experience for their game level. The challenge entity contains how many experience points (XP) are given upon completion and the challenge description. Whether a challenge is completed or not for each user is also tracked as an attribute of the relationship between user account and challenge.

File Use

Frontend Files:

The project's frontend systems will be developed using React, which is mostly written using JavaScript. This will handle all user interactions (buttons, page navigation, etc.) as well as any interface updates. On the other hand, the visual layout and styling are defined by CSS and SCSS. For images and icons used, we use a PNG for the logo and favicon (tab icon), JPG for user profile pictures, and SVG for icons used on the page for buttons.

Backend/Database Files:

Most of the backend logic is also developed in JavaScript. Node.js is used for managing database communication and API interaction. EJS was also used to render server-side templates dynamically.

Configuration and Dependency Files:

- *config.json* defines the database connection settings that tell the backend how to connect to the MySQL server under different environments (development, testing, and production).
- *package.json* defines the project's metadata, dependencies, and scripts for running and maintaining the app. It lists all the packages required for both frontend and backend components such as React, Socket.io, Axios, and Agora.io. It also defines startup commands and development dependencies.

Environment File:

- *.env* stores sensitive information such as API keys and database credentials

Data Exchange

The app talks to the server in two ways. For most of what users do (logging in, signing up, editing their profile, managing friends and meetings, or uploading and loading transcripts and recordings), the React frontend uses Axios to send HTTP requests to the Node.js backend. In addition, everything is going back and forth in JSON. For the live chat, the frontend keeps an open connection to a separate Socket.io server and sends and receives messages through events so they show up right away.

The backend is where data actually lives, and it uses a MySQL connection pool and runs the SQL that adds, changes, fetches, and deletes records. For the AI side, the backend talks to the model through an MCP server, sends it prompts, and then plugs the model's replies into the AI assistant and chat. Furthermore, when users send voice to the assistant or upload recordings, the frontend sends that multipart form data, and the backend either processes it in memory or saves it.

Format and Protocols

The frontend, backend, and database exchange is in JSON. For normal web traffic, the client talks to the server over HTTP (or HTTPS in production) using a REST-style API. Most of the endpoints live under `/api/v1/`, and a few older ones sit at the root, like `/api/login` and `/Register`. Real-time chat runs over Socket.io instead of HTTP, so messages can go back and forth without repeated requests. The AI features use MCP for

talking to the model, so the backend sends prompts and gets back the responses that show up in the assistant and chat. When users upload audio for the AI assistant or for recordings, the frontend sends it as multipart form data. In addition, the server only accepts certain audio and video types and enforces a size limit on those uploads.

Security Considerations

The app collects names, emails, and profile information, so how that data is stored and sent matters. Passwords get the most attention, so they're hashed with Bcrypt before being saved, and the backend never keeps or checks plain-text passwords. Users sign in with their email and password, and the app handles authentication itself instead of using a separate login service. The app doesn't handle any financial data. Other details like names, emails, and profile fields also live in MySQL, and users must be logged in to use matchmaking, chat, their profile, or the AI assistant.

The backend checks that the person making each request is allowed to see or change the data before it responds. The live site runs on Georgia Tech's Plesk Hosting at <https://languagematchmaker.modlangs.gatech.edu>. Plesk is a shared hosting service run by OIT; it uses a Web Application Firewall for web traffic and is monitored around the clock. Getting to the app also requires Global Protect VPN and a Georgia Tech login, so only people who are supposed to use it can reach it. In that setup, the frontend and backend talk over HTTPS so traffic is encrypted. AI features go through the backend, which decides who can start or use an AI session. CORS is set so only the real frontend origin can call the API, which helps keep other sites from using it. Profile and chat data are only shown to the user they belong to or to people the app is designed to share with (like matched partners), and the backend enforces that.

